

Task 2

COMPUTATIONAL COMPLEXITY ANALYSIS OF A CONSTRAINT-BASED SEATING PROBLEM

Understanding the Problem Through P vs NP Classification

The classroom seating arrangement scenario represents a classic example of a constraint-based combinatorial problem. In this situation, a teacher must arrange students in seats while satisfying specific constraints, such as preventing certain students from sitting next to each other, ensuring fairness, or meeting predefined grouping conditions. Although the task appears simple at a small scale, its complexity increases significantly as the number of students grows. To analyse this problem properly, it must be evaluated using the framework of computational complexity theory, particularly the distinction between P and NP problems.

A problem belongs to class P (Polynomial Time) if it can be solved efficiently using an algorithm ([Sipser, 2013](#)) whose running time grows polynomially with input size. In simpler terms, as the number of students increases, the time required to compute a seating arrangement grows at a manageable rate. Sorting numbers or searching within a structured list are typical examples of P problems.

In contrast, a problem belongs to class **NP (Nondeterministic Polynomial Time)** if a proposed solution can be verified quickly, but finding that solution may require extensive computation. This is a crucial distinction. For the seating arrangement scenario, once someone provides a completed seating plan, checking whether all constraints are satisfied is relatively straightforward. One can simply examine each pair of seats and verify whether the rules are followed. This verification step can be completed in polynomial time.

However, generating a valid arrangement from scratch is far more demanding. If there are n students, there are $n!$ (factorial) possible seating arrangements. Factorial growth increases extremely rapidly. For example:

- 5 students $\rightarrow$ 120 possible arrangements

- 10 students → 3,628,800 arrangements
- 20 students → over 2.4 quintillion arrangements

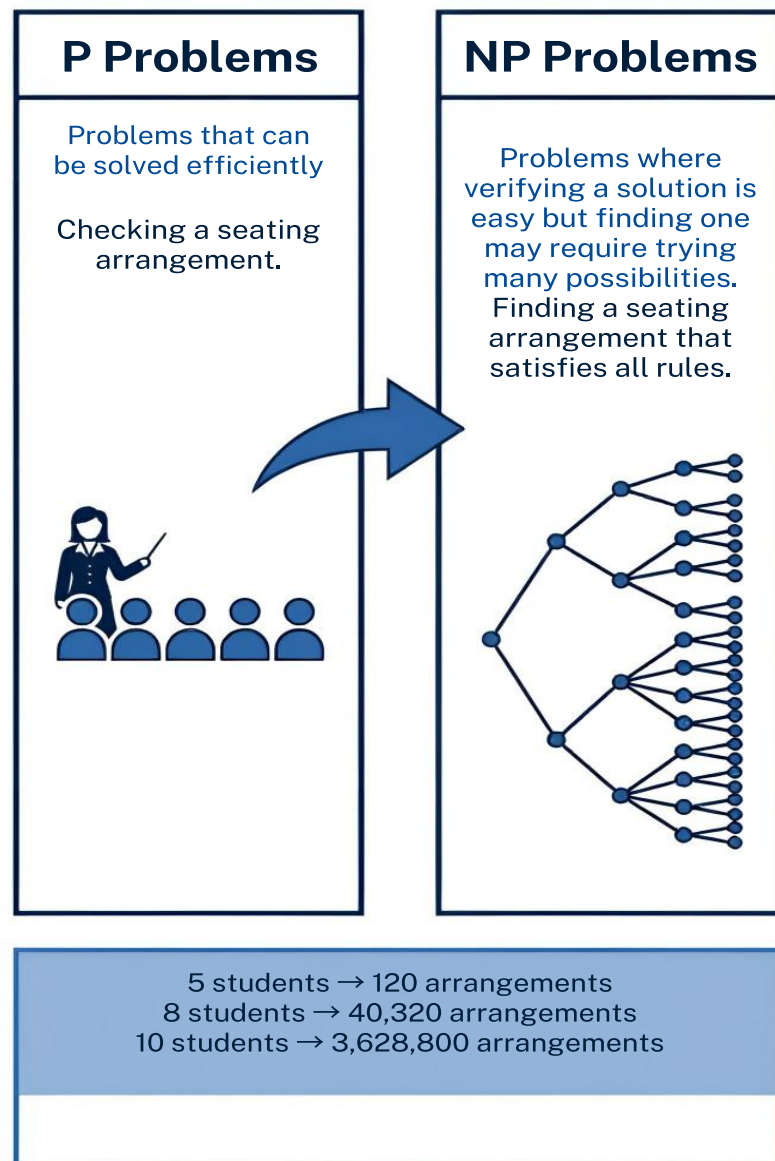


Figure 1: Factorial growth makes brute-force search impractical

As illustrated in Figure 5, the number of possible seating arrangements increases factorially with the number of students, making brute force methods impractical for larger inputs. This explosive growth illustrates why brute computational methods become infeasible as the input size increases. Because verifying a solution is easy but finding a solution may require exploring an exponential number of possibilities, the seating arrangement problem aligns closely with NP-

type combinatorial problems. It resembles well-known NP-complete problems such as the graph colouring problem or constraint satisfaction problems, where constraints must be satisfied simultaneously.

Therefore, from a computational complexity perspective, the classroom seating arrangement problem can be classified as an NP-type combinatorial optimisation problem. This classification explains why the problem becomes computationally expensive as scale increases and justifies the need for alternative solution strategies beyond simple enumeration.

Brute Force Approach

A straightforward way to solve the seating arrangement problem is through a brute force method. The brute force strategy attempts to generate all possible seating arrangements and then checks each arrangement against the given constraints. The first valid arrangement found is selected as the solution. Conceptually, this approach is simple and guarantees correctness because every possible configuration is examined. If a valid arrangement exists, brute force will eventually find it. However, its practicality depends entirely on the number of students involved. The number of seating arrangements grows factorially as the number of students increases ([Cormen, 2009](#)).

The time complexity of brute force in this scenario is factorial, represented as $O(n!)$. Factorial complexity grows much faster than polynomial complexity. This means that even small increases in input size result in dramatic increases in computation time. For example, with 6 students, there are 720 possible arrangements. A computer can evaluate these quickly. However, with 15 students, the number increases to over 1.3 trillion arrangements. At that scale, even modern computers would struggle to process all possibilities within reasonable time limits. The brute force approach demonstrates an important theoretical concept: correctness does not imply efficiency. While brute force is guaranteed to find a solution if one exists, it becomes computationally infeasible as problem size increases. Another limitation of brute force is resource consumption. As factorial growth accelerates, memory and processing requirements increase dramatically. In real-world systems, such exponential growth makes brute force impractical beyond small datasets.

Therefore, although brute force provides a foundational understanding of the problem structure and guarantees completeness, it cannot be considered an efficient or scalable solution for large classroom scenarios.

Heuristic (Smart) Approach

Because brute force becomes impractical for larger inputs, heuristic approaches are introduced as more efficient alternatives. A heuristic is a strategy designed to produce a good solution within reasonable time, even if it does not guarantee the optimal solution in every case.

In the classroom seating scenario, a heuristic approach might involve:

- Placing students with strict constraints first
- Grouping compatible students together early
- Using graph-based modelling to represent conflicts
- Applying greedy algorithms to assign seats sequentially

For example, students who must not sit together can be represented as connected nodes in a graph. The problem then becomes similar to a graph colouring problem, where adjacent nodes must not share the same category. By using structured placement rules, the search space is significantly reduced compared to exploring all permutations. Heuristic methods typically operate in polynomial or near-polynomial time. Instead of examining $n!$ arrangements, the algorithm narrows the search using logical rules, constraint propagation, or prioritisation techniques ([Russell and Norvig, 2011](#)). The primary advantage of heuristics is efficiency. They dramatically reduce computational time and make large-scale problems manageable. In practical educational settings, where time is limited, heuristics provide realistic and implementable solutions.

However, heuristics come with limitations. They may not always produce the optimal arrangement, and in rare cases, they may fail to find a valid configuration even when one exists. This trade-off between optimality and efficiency is central to computational complexity theory. Despite this limitation, heuristics are widely used in real-world applications such as scheduling, resource allocation, and routing problems. Their effectiveness demonstrates that practical problem-solving often prioritises feasibility over theoretical optimality.

In conclusion, while the seating arrangement problem aligns with NP-type complexity, heuristic strategies provide a balanced approach by reducing computational cost while maintaining acceptable solution quality.

CRITICAL DISCUSSION

The seating arrangement problem illustrates a fundamental concept in computer science: the gap between theoretical solvability and practical feasibility.

While the problem is solvable in theory through brute force, factorial growth makes exhaustive search impractical at scale. The distinction between verifying and solving highlights why NP-type problems are central to complexity theory.

In real-world systems such as scheduling, timetabling, routing, and resource allocation, similar constraint-based problems occur frequently. Heuristic approaches are widely adopted because they provide acceptable solutions within reasonable time limits.

Therefore, although the seating arrangement problem aligns with NP-type complexity, practical strategies focus on reducing search space rather than achieving guaranteed optimal solutions.

CONCLUSION

The classroom seating arrangement problem demonstrates how computational complexity theory applies to everyday decision-making scenarios. While verifying a seating plan can be performed efficiently in linear time, generating a valid arrangement requires exploring factorial possibilities, placing the problem closer to the NP category.

The brute force method guarantees correctness but fails to scale as student numbers increase. Heuristic approaches provide a more practical alternative by reducing computational effort, even though they may sacrifice guaranteed optimality.

This analysis illustrates the importance of understanding computational complexity when designing algorithms. Efficient system design requires balancing correctness, scalability, and practicality.